CS543 – WEB SERVICES – ASSIGNMENT 4

Service Selected: RAG (Product Verdict + Chat)

Repository: https://github.com/udaygupta9081/shop-pilot-ai/tree/master (master branch only)

Evidence note: The repository source could not be fetched for direct inspection in this session. The status labels below are based on the uploaded draft and the user-provided scope: /products/verdict is implemented; chat and message operations are not yet implemented. Any additional API design is explicitly marked as proposed, not as existing code.

Assignment-fit note: The assignment brief asks students to choose Catalogue, Orders, or Delivery from the CampusEats service and says not to choose Payments. RAG (Product Verdict + Chat) is outside those named choices. Confirm with the instructor that this alternative repository/service is acceptable before submission.

Service Overview

The selected service is intended to retrieve product-related data, store/retrieve information using a vector database, and answer user queries. The original draft names Price Agent, Verdict Agent, and Chat Agent as components/conversation models. Only the /products/verdict endpoint is identified as implemented in the supplied project context. The Price Agent and Chat Agent workflows, plus chat/message persistence and conversation retrieval endpoints, are not confirmed as implemented.

Component / capability	Status based on available evidence	Notes
Product Verdict RAG endpoint	Implemented (user-reported)	Existing draft shows GET /products/verdict with product_id and product_name query parameters. Exact response schema and error behavior need source-code verification.
Price Agent	Not verified	Mentioned in the service description, but no implementation details or endpoint were supplied.
Chat Agent / conversation flow	Yet to be implemented (user-reported)	Conversation and message operations are listed as planned SOAP/REST operations, not completed functionality.
Vector database ingestion/retrieval	Purpose described; details not verified	Do not claim a specific vector database, embedding model, collection, or ingestion pipeline without inspecting the repository.

PART A – Model the Service

A1. Selected Service

Selected service: RAG (Product Verdict + Chat). It is intended to answer product-verdict queries using retrieved information and to support conversational interactions. Important: this does not match the three service choices specified in the assignment PDF (Catalogue, Orders, or Delivery). Instructor approval is needed if the repository choice is to be retained.

A2. SOAP Operations

The following SOAP-style operation names were in the original draft. They describe intended operations and must not be mistaken for implemented endpoints.

SOAP operation	Purpose	Implementation status
CreateChat()	Create a conversation for a user.	Yet to be implemented
GetAllChats()	List a user's conversations.	Yet to be implemented
GetChat()	Retrieve one conversation.	Yet to be implemented
CreateMessage()	Add a message to a conversation.	Yet to be implemented
GetAllChatMessages()	Retrieve messages in a conversation.	Yet to be implemented

updateMessage()	Update an existing message.	Yet to be implemented / verified
GetVerdict()	Retrieve a product verdict.	Corresponding REST route /products/verdict is user-reported as implemented

A3. Find the Nouns / Resource Mapping

REST URLs should name resources rather than repeat verbs such as Create, Get, or Update. The paths below are a cleaned-up proposed mapping based on the original draft; only /products/verdict is reported implemented.

SOAP operation	Proposed REST resource	What it represents	Status
CreateChat()	POST /conversation/user/{userId}	Create a conversation for a user.	not implemented
GetAllChats()	GET /conversation/user/{userId}	List conversations for a user.	not implemented
GetChat()	GET /conversation/{chatId}	Read one conversation.	not implemented
CreateMessage()	POST /conversation/{chatId}/messages	Create a message within a conversation.	not implemented
GetAllChatMessages()	GET /conversation/{chatId}/messages	List messages in a conversation.	not implemented
updateMessage()	PUT /messages/{messageId}	Replace/update a message resource.	not implemented
GetVerdict()	GET /products/verdict	Return a verdict for the supplied product.	Implemented

A4. Resource Table

The table distinguishes the endpoint already identified in the project from the endpoints proposed for the unfinished chat workflow. Status codes for proposed routes are design choices, not observed runtime behavior.

Method	URL	What it does	Success code	Failure codes / status
GET	/products/verdict?product_id={id}&product_name={name}	Returns a product verdict using the product identifiers supplied as query parameters.	200 (expected for a read; verify actual code)	400 for invalid/missing query values; 404 if product/evidence is not found; 500 for unexpected failure — verify implementation
POST	/conversation/user/{userId}	Creates a conversation for a user.	201	400 malformed input; 401/403 unauthorized; 404 user not found; 409 conflict
GET	/conversation/user/{userId}	Lists a user's conversations.	200	401/403 unauthorized; 404 user not found
GET	/conversation/{chatId}	Retrieves one conversation.	200	401/403 unauthorized; 404 conversation not found
POST	/conversation/{chatId}/messages	Adds a message to a conversation and may invoke the chat/RAG workflow.	201	400 malformed input; 401/403 unauthorized; 404 conversation not found; 422 invalid domain request
GET	/conversation/{chatId}/messages	Retrieves messages belonging to a conversation.	200	401/403 unauthorized; 404 conversation not found
PUT	/messages/{messageId}	Updates a message.	200	400 malformed input; 401/403 unauthorized; 404 message not found; 409 conflicting state

A5. Justification of the Difficult Choice

The difficult design choice is how to keep a conversation's history available when a new message is submitted to the RAG/LLM workflow. The original draft identifies the need to validate the user, create and store a conversation, and provide prior messages to the model so that later answers retain context. A proposed design is to treat conversations and messages as separate resources and retrieve the conversation's relevant history before generating a response. The trade-off is between sending the entire history, which can increase request size and processing cost, and selecting a bounded/relevant history,

which requires a clear context-selection strategy. This is a design explanation only; the chat workflow is not reported as implemented yet.

PART B – Publish the Contract

B1. OpenAPI Contract

Status: an OpenAPI file was not supplied with the uploaded draft, and the repository could not be inspected. Therefore, the YAML below is a proposed starter contract for the implemented /products/verdict route only. It must be reconciled with the actual handler and expanded when the chat routes are implemented. It is not evidence that openapi.yaml currently exists.

```
openapi: 3.0.3
info:
  title: Shop Pilot AI - Product Verdict API
  version: 0.1.0
  description: Proposed contract based on the supplied assignment draft.
servers:
  - url: http://127.0.0.1:8000
paths:
  /products/verdict:
    get:
      summary: Retrieve a product verdict
      description: Query parameter names are taken from the supplied draft; response schema must be
confirmed against the handler.
      parameters:
        - name: product_id
          in: query
          required: true
          schema:
            type: string
        - name: product_name
          in: query
          required: true
          schema:
            type: string
      responses:
        '200':
          description: Verdict generated/retrieved successfully
          content:
            application/json:
              schema:
                $ref: '#/components/schemas/VerdictResponse'
        '400':
          description: Missing or invalid query parameters
          content:
            application/json:
              schema:
                $ref: '#/components/schemas/Problem'
        '404':
          description: Product or supporting evidence not found
          content:
            application/json:
              schema:
                $ref: '#/components/schemas/Problem'
        '500':
          description: Unexpected server error
          content:
            application/json:
              schema:
                $ref: '#/components/schemas/Problem'
components:
  schemas:
    VerdictResponse:
      type: object
      description: Placeholder; replace with the actual response fields returned by the implementation.
      additionalProperties: true
    Problem:
      type: object
      required: [type, title, status, detail]
      properties:
        type:
          type: string
        title:
```

```
      type: string
    status:
      type: integer
    detail:
      type: string
```

Important: VerdictResponse is intentionally a placeholder because the real response shape was not available to inspect. Do not submit this contract as validated until the schema matches the code and an OpenAPI validator reports zero errors.

B2. Define Schemas Once

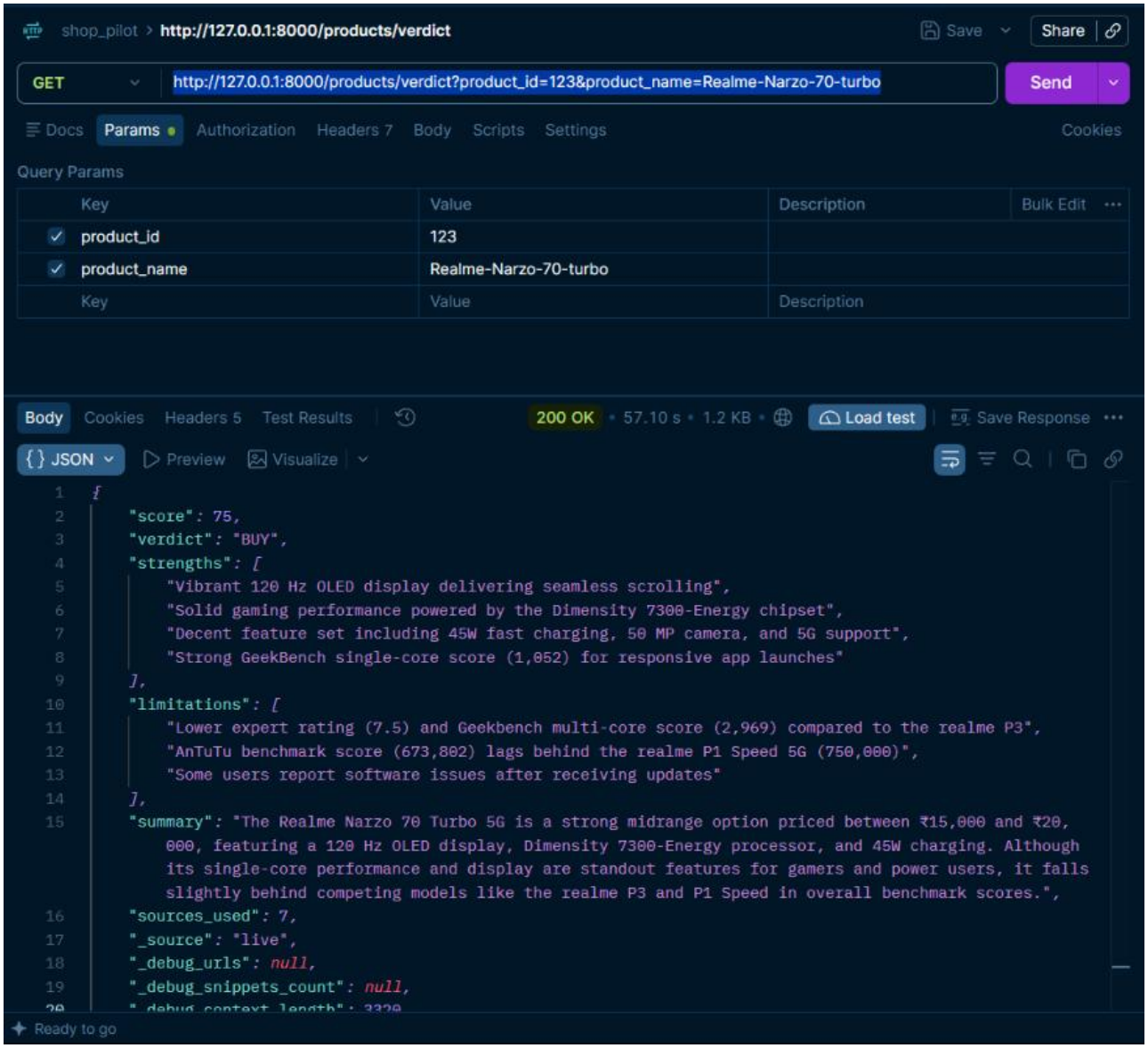
The proposed YAML defines VerdictResponse and Problem under components.schemas and references them with $ref. The response schema must be updated to match the actual JSON returned by /products/verdict. Request/response schemas for conversations and messages should be added only when their data fields and handlers are defined.

B3. Declare Every Failure

The proposed /products/verdict contract documents 400, 404, and 500 failure responses. These are proposed contract outcomes, not verified behavior. Inspect the route to determine its actual status codes and error body, then make the OpenAPI responses match the implementation. All future chat endpoints must document their own failure paths as well.

B4. Validate the Contract

/product /verdict is implemented till now



PART C – Implement It

Refer Github (RAG Folder)